

ECE 220 Computer Systems & Programming

Lecture 7 — Introduction to Functions in C

September 15, 2026

Prof. Sayan Mitra `mitras@illinois.edu`

Course website: `courses.grainger.illinois.edu/ece220/fa2026`

I ILLINOIS

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

Functions in C

**Subroutines are back — with names, types,
and no register juggling.**

```
; arg n in R0 (by convention)
; save R7!  callee-save R3!
      JSR  FACT
; result in R2 (by convention)
```

LC-3 subroutine call (Lecture 2)



```
answer = Fact(n);
```

C function call (today)

Today's Two Questions

**A function is a contract:
typed inputs, one typed output.**

1. How does data get *into* and *out of* a C function?
2. What happens to `main`'s variables while `Fact` runs?

Prototype, Call, Definition

```
/* answer = Fact(number): prototype, call, definition */
#include <stdio.h>

int Fact(int n);           /* PROTOTYPE: ends with ; */

int main() {
    int number, answer;
    printf("Enter a number: ");
    scanf("%d", &number);
    answer = Fact(number);  /* the CALL */
    printf("factorial of %d is %d\n", number, answer);
    return 0;
}

int Fact(int n) {          /* the DEFINITION */
    int i, result = 1;     /* locals: born at the call */
    for (i = 1; i <= n; i++)
        result = result * i;
    return result;         /* typed return value */
}
```

- **Prototype:** tells the compiler the types — of the arguments and the return value — before the first call; ends with **;**
- **Call:** `Fact(number)` is an *expression* with a value
- **Definition:** the code; may appear before or after `main` (before: no prototype needed)

Name, return type, parameter types — read
`int Fact(int n)` **inside-out**.

Inside the Definition

```
int Fact(int n) {                /* the DEFINITION */
    int i, result = 1;           /* locals: born at
    the call */
    for (i = 1; i <= n; i++)
        result = result * i;
    return result;               /* typed return value
    */
}
```

- `n`, `i`, `result` are **local**: born at the call, gone after `return`
- `return` does two jobs: hands back a value *and* ends the function
- What is `i` after the return?
gone — locals die at return

In-class: what does `Fact(4)` return? 24

Calls Inside Calls

```
int main() {  
    double a = 3.0, b = 4.0, hyp;  
    int n = 5, k = 2, ways;  
  
    /* inner calls finish first */  
    hyp = sqrt(pow(a, 2) + pow(b, 2));  
    /* Fact runs three times */  
    ways = Fact(n) / (Fact(k) * Fact(n - k));  
  
    printf("hyp = %.1f, ways = %d\n", hyp, ways);  
    return 0;  
}
```

In-class, with $n = 5$, $k = 2$:

Fact runs 3 times, and ways = 10

- An argument can be any expression
— *even another call*

`sqrt` and `pow` come from the math library: `#include <math.h>`, then link with `-lm` — the linker from Lecture 5.

Calls Inside Calls

```
int main() {  
    double a = 3.0, b = 4.0, hyp;  
    int n = 5, k = 2, ways;  
  
    /* inner calls finish first */  
    hyp = sqrt(pow(a, 2) + pow(b, 2));  
    /* Fact runs three times */  
    ways = Fact(n) / (Fact(k) * Fact(n - k));  
  
    printf("hyp = %.1f, ways = %d\n", hyp, ways);  
    return 0;  
}
```

In-class, with $n = 5$, $k = 2$:

Fact runs 3 times, and ways = 10

`sqrt` and `pow` come from the math library: `#include <math.h>`, then link with `-lm` — the linker from Lecture 5.

- An argument can be any expression — *even another call*
- The inner call finishes first; its return value becomes the outer call's argument

Calls Inside Calls

```
int main() {  
    double a = 3.0, b = 4.0, hyp;  
    int n = 5, k = 2, ways;  
  
    /* inner calls finish first */  
    hyp = sqrt(pow(a, 2) + pow(b, 2));  
    /* Fact runs three times */  
    ways = Fact(n) / (Fact(k) * Fact(n - k));  
  
    printf("hyp = %.1f, ways = %d\n", hyp, ways);  
    return 0;  
}
```

In-class, with `n = 5`, `k = 2`:

Fact runs 3 times, and ways = 10

`sqrt` and `pow` come from the math library: `#include <math.h>`, then link with `-lm` — the linker from Lecture 5.

- An argument can be any expression — *even another call*
- The inner call finishes first; its return value becomes the outer call's argument
- Every call gets its own fresh `n`, `i`, `result` — even three calls on one line

Calls Inside Calls

```
int main() {  
    double a = 3.0, b = 4.0, hyp;  
    int n = 5, k = 2, ways;  
  
    /* inner calls finish first */  
    hyp = sqrt(pow(a, 2) + pow(b, 2));  
    /* Fact runs three times */  
    ways = Fact(n) / (Fact(k) * Fact(n - k));  
  
    printf("hyp = %.1f, ways = %d\n", hyp, ways);  
    return 0;  
}
```

In-class, with `n = 5`, `k = 2`:

`Fact` runs 3 times, and `ways = 10`

`sqrt` and `pow` come from the math library: `#include <math.h>`, then link with `-lm` — the linker from Lecture 5.

- An argument can be any expression — *even another call*
- The inner call finishes first; its return value becomes the outer call's argument
- Every call gets its own fresh `n`, `i`, `result` — even three calls on one line
- C does *not* promise which argument is evaluated first. Harmless here: `Fact` changes nothing outside itself

Calls Inside Calls

```
int main() {  
    double a = 3.0, b = 4.0, hyp;  
    int n = 5, k = 2, ways;  
  
    /* inner calls finish first */  
    hyp = sqrt(pow(a, 2) + pow(b, 2));  
    /* Fact runs three times */  
    ways = Fact(n) / (Fact(k) * Fact(n - k));  
  
    printf("hyp = %.1f, ways = %d\n", hyp, ways);  
    return 0;  
}
```

In-class, with $n = 5$, $k = 2$:

Fact runs 3 times, and $ways = \underline{10}$

Three calls, three sets of locals. Next lecture: where they live.

`sqrt` and `pow` come from the math library: `#include <math.h>`, then link with `-lm` — the linker from Lecture 5.

- An argument can be any expression — *even another call*
- The inner call finishes first; its return value becomes the outer call's argument
- Every call gets its own fresh `n`, `i`, `result` — even three calls on one line
- C does *not* promise which argument is evaluated first. Harmless here: `Fact` changes nothing outside itself

Functions You Already Use

- `printf`, `scanf`, `getchar` — library functions, declared in `<stdio.h>`
- They *return values* you have been ignoring: `scanf` returns the number of values it read — checking it catches bad input
- `void` return type = nothing comes back; `f(void)` or `f()` = nothing goes in
- `man 3 printf` shows any library function's full contract

Every function call in every C program follows today's rules — including `main` itself, called by the operating system.

Arguments Are Copies (Pass by Value)

```
/* Arguments are COPIES (pass by value) */
#include <stdio.h>

void tryToChange(int a) {
    a = 99;                      /* changes the copy */
    /*
    printf("inside:  a = %d\n", a);
    */
}

int main() {
    int x = 5;
    tryToChange(x);
    printf("outside: x = %d\n", x);
    return 0;
}
```

In-class — predict the output:

inside: a = 99

outside: x = 5 — unchanged!

- The call copies the *value* of `x` into a brand new variable `a`
- Nothing the callee does to its copy can touch the caller's original

Why Bother? Decomposition

One menu program — a size, then `s` for a box of stars or `h` for hashes — written two ways:

Without functions

```
switch (k) {  
  case 's':  
    for (r = 0; r < n; r++) {  
      for (c = 0; c < n; c++)  
        putchar('*');  
      putchar('\n');  
    }  
    break;  
  case 'h':  
    for (r = 0; r < n; r++) {  
      for (c = 0; c < n; c++)  
        putchar('#');  
      putchar('\n');  
    }  
    break;  
}
```

Change the printing? Edit *both* copies.

Why Bother? Decomposition

One menu program — a size, then `s` for a box of stars or `h` for hashes — written two ways:

Without functions

```
switch (k) {  
  case 's':  
    for (r = 0; r < n; r++) {  
      for (c = 0; c < n; c++)  
        putchar('*');  
      putchar('\n');  
    }  
    break;  
  case 'h':  
    for (r = 0; r < n; r++) {  
      for (c = 0; c < n; c++)  
        putchar('#');  
      putchar('\n');  
    }  
    break;  
}
```

With a function

```
void box(int n, char ch) {  
  int r, c;  
  for (r = 0; r < n; r++) {  
    for (c = 0; c < n; c++)  
      putchar(ch);  
    putchar('\n');  
  }  
}
```

```
switch (k) {  
  case 's': box(n, '*'); break;  
  case 'h': box(n, '#'); break;  
}
```

One loop, one name; the parameter carries what differs.

Change the printing? Edit *both* copies.

Same reasons we wrote LC-3 subroutines — now the compiler checks the contract.

In-Class Problem: Write swap

Fill in `swap` so it exchanges `x` and `y` — then predict the output.

- Does it print `x = 3, y = 2`?
No! `x = 2, y = 3`
- Why? `swap` exchanged its own *copies* — `main`'s `x` and `y` never moved.

To fix `swap` we must find out where `x` and `y` actually *live*. Thursday: the run-time stack.

```
/* Does this swap main's x and y? */
#include <stdio.h>

void swap(int x, int y) {
    int temp;
    temp = x;
    x = y;
    y = temp;
}

int main() {
    int x = 2, y = 3;
    swap(x, y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

Summary & What's Next

Today

- **Anatomy:** prototype (types, before first use), call (an expression), definition (the code)
- **The contract:** zero or more typed arguments in, at most one typed value out; locals are born at the call and die at return
- **Pass by value:** arguments are *copies* — which is why `swap` silently fails

Next lecture

- Where the copies live: activation records, R5/R6, and the full C calling convention in LC-3

Code from today: `fact.c`, `compose.c`, `copy.c`, `swap.c`, `menu_nofun.c`, `menu_fun.c`.